

Pydantic

Pydantic is one of the **most important libraries in modern Python backend development**, especially with FastAPI.

It helps you:

- Validate data automatically
- Write cleaner code
- Prevent errors
- Build reliable APIs

If FastAPI is the **engine**, then Pydantic is the **safety system + data manager** behind it.

What You'll Learn

In this lesson, we'll cover:

- What Pydantic is
 - Why it is useful
 - How it works
 - Field Validators
 - Model Validators
 - Computed Fields
 - Nested Models
 - Serialization
-

1. Intro

The lesson introduces:

- Pydantic basics
 - Data validation
 - Type safety
 - Clean data modeling
-

What Problem Does Pydantic Solve?

Imagine users send data to your API.

Some users may send:

```
{  
  "name": "Ali",  
  "age": "twenty"  
}
```

But your application expects:

age = integer

Without validation:

Your program may crash

Pydantic fixes this automatically.

It:

- Checks data types
 - Validates inputs
 - Gives clear errors
 - Converts data when possible
-

2. Why is Pydantic Used?

This is the most important conceptual section.

Simple Definition

Pydantic is a Python library used to define and validate structured data using Python type hints.

Real-Life Analogy

Imagine airport security.

Passengers = incoming data

Security rules = validation

Allowed passengers = valid data

Pydantic checks:

- Is the data correct?
 - Is required information present?
 - Are types valid?
-

Main Reasons Pydantic is Used

1 Automatic Data Validation

Example:

```
from pydantic import BaseModel

class Student(BaseModel):
    name: str
    age: int
```

Valid Input

```
student = Student(name="Ali", age=20)
```

Works perfectly.

Invalid Input

```
student = Student(name="Ali", age="twenty")
```

Pydantic raises error automatically.

2 Cleaner Code

Without Pydantic:

```
if not isinstance(age, int):  
    raise ValueError()
```

You manually validate everything.

With Pydantic:

Validation happens automatically.

3 Type Safety

Pydantic uses Python type hints.

Example:

```
name: str  
age: int
```

This clearly defines expected data.

4 Perfect for APIs

APIs receive user data constantly.

Pydantic ensures:

- Correct format
- Safe inputs
- Reliable backend logic

That's why FastAPI uses Pydantic heavily.

5 Automatic Parsing

Pydantic can convert types automatically.

Example:

```
Student(name="Ali", age="20")
```

Pydantic converts:

"20" → 20

3. How Does Pydantic Work?

Now we learn the internal concept.

Step 1: Create Model

Pydantic models inherit from:

BaseModel

Example:

```
from pydantic import BaseModel
```

```
class User(BaseModel):
```

```
    name: str
```

```
    age: int
```

Step 2: Create Object

```
user = User(name="Ahmed", age=22)
```

What Pydantic Does Internally

When object is created:

1. Reads type hints
 2. Checks input
 3. Validates data
 4. Converts compatible values
 5. Raises errors if invalid
-

Example Error

```
User(name="Ali", age="hello")
```

Error:

Input should be a valid integer

Important Concept: Schema

A schema means:

Structure/rules of data

Example schema:

- name = string
- age = integer
- email = string

Pydantic models define schemas.

Optional Fields

```
from typing import Optional
```

```
class User(BaseModel):  
    name: str  
    age: Optional[int] = None
```

Meaning:

Age is optional.

Default Values

```
class User(BaseModel):  
    country: str = "Pakistan"
```

If no country given:
default becomes "Pakistan"

4. Field Validator

Field validators validate specific fields.

Purpose

Used when:

- Normal type checking is not enough
 - You want custom rules
-

Example

```
from pydantic import BaseModel, field_validator  
  
class User(BaseModel):  
    age: int  
  
    @field_validator("age")
```

```
def validate_age(cls, value):  
    if value < 18:  
        raise ValueError("Age must be at least 18")  
    return value
```

Valid

```
User(age=25)
```

Invalid

```
User(age=15)
```

Error:

Age must be at least 18

Real-World Uses

Validators are useful for:

- Password rules
- Age limits
- Email checking
- Input restrictions

5. Model Validator

Field validator checks one field.

Model validator checks:

Entire model together

Example Problem

Suppose:

- password
- confirm_password

Both must match.

Example

```
from pydantic import BaseModel, model_validator  
  
class User(BaseModel):  
    password: str  
    confirm_password: str
```

```
@model_validator(mode="after")
def passwords_match(self):
    if self.password != self.confirm_password:
        raise ValueError("Passwords do not match")
    return self
```

Why Needed?

Because validation depends on multiple fields together.

6. Computed Fields

Computed fields are values calculated automatically.

Example

Suppose:

- first_name
- last_name

You want:

- full_name
-

Example

```
from pydantic import BaseModel, computed_field

class User(BaseModel):
    first_name: str
    last_name: str

    @computed_field
    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"
```

Output

```
{
  "first_name": "Ali",
  "last_name": "Khan",
  "full_name": "Ali Khan"
}
```

```
}
```

Benefit

Avoids recalculating values manually.

7. Nested Models

Nested models mean:

Models inside other models

Example

Student has address.

Example

```
class Address(BaseModel):
    city: str
    country: str

class Student(BaseModel):
    name: str
    address: Address
```

Input

```
student = Student(
    name="Ali",
    address={
        "city": "Abbottabad",
        "country": "Pakistan"
    }
)
```

Why Important?

Real applications have structured data:

- User → Address
- Order → Product
- Blog → Comments

Nested models organize this cleanly.

8. Serialization

Serialization means:

Converting Python objects into JSON/dictionary format

Why Needed?

APIs usually send JSON responses.

Example

```
user.model_dump()
```

Output

```
{  
  "name": "Ali",  
  "age": 20  
}
```

Convert to JSON

```
user.model_dump_json()
```

Simple Understanding

Serialization:

Python Object → JSON

Deserialization:

JSON → Python Object

Real FastAPI Connection

When FastAPI receives request:

1. Pydantic validates request data
 2. Converts data into Python object
 3. FastAPI processes it
 4. Response serialized into JSON
-

Complete Mental Model

Without Pydantic

You manually:

- Check types
-

- Validate inputs
- Handle errors
- Convert data

More code

More bugs

With Pydantic

Pydantic automatically:

Validates

Parses

Converts

Structures data

Handles errors

Important Pydantic Concepts Summary

Concept	Purpose
BaseModel	Create schema
Field Validator	Validate single field
Model Validator	Validate entire model
Computed Field	Auto-generated value
Nested Model	Structured complex data
Serialization	Convert object to JSON

Memory Trick

Pydantic = “Smart Data Security Guard”

It checks:

- Type
- Format
- Rules
- Structure

Before data enters your application.

Why This Matters for Machine Learning APIs

When deploying ML models:

- Users send inputs
- Pydantic validates them
- FastAPI safely processes requests

Example:

```
{  
  "text": "This movie is amazing!"  
}
```

Pydantic ensures:

- text exists
 - type is correct
 - invalid requests are rejected
-

Final Takeaway

Pydantic is essential because it:

- Makes APIs reliable
- Reduces bugs
- Simplifies validation
- Keeps code clean
- Works perfectly with FastAPI